

AIOps Assignment 2 — Infrastructure & Containerization

Question 1 - Single-Stage vs. Multi-Stage Docker

Reduction: approximately 67.56%

IMAGE	DISK USAGE	CONTENT SIZE
spam-api-single:latest	2.09GB	520MB
spam-api-multi:latest	678MB	155MB

Explanation:

The multi-stage image is smaller because the final stage uses the lightweight `python:3.11-slim` base image instead of the full `python:3.11` base used in the single-stage image, and `pip`'s download cache is not retained because `--no-cache-dir` is used. The builder-only `/build` directory containing `requirements.txt` and the builder stage's filesystem/layers are left behind, while only the installed dependencies from `/install` are copied into the final image.

Question 2 - Multi-Container Orchestration with Docker Compose

The API was extended to check a Redis cache before running inference. On a cache miss, the prediction is computed and stored in Redis with a 300-second TTL; on a cache hit, the cached label is returned directly without invoking the model. The two services (`api` and `cache`, using the official `redis:7-alpine` image) were defined in `docker-compose.yml` and connected over Compose's default network, with the API reaching Redis via the service name `cache` as its hostname, configured through the `REDIS_HOST` environment variable.

Evidence of cache-hit speedup :

```
$ docker compose ps
NAME                IMAGE                SERVICE  STATUS      PORTS
q2-api-1            q2-api              api      Up 3 hours  0.0.0.0:5000->5000/tcp
q2-cache-1          redis:7-alpine       cache    Up 3 hours  0.0.0.0:6379->6379/tcp

$ time curl -s -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "WIN a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"cached":false,"label":"spam"}
real    0m1.334s

$ time curl -s -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"text": "WIN a FREE iPhone now! Click here: bit.ly/xyz123"}'
{"cached":true,"label":"spam"}
real    0m0.148s
```

The first request is a cache miss (`cached: false`, 1.334s); the identical second request is a cache hit (`cached: true`, 0.148s) it's roughly a 9x speedup (approximately 89% reduction in response time).

Compose vs. Kubernetes: Docker Compose solves multi-container orchestration on a single host defining how a fixed set of containers network, start, and depend on each other on one machine. Kubernetes operates at cluster scale across multiple nodes, handling scheduling, self-healing, and rolling updates that Compose doesn't do.

Question 3 — Kubernetes Indexed Job: Parallel Data Validation

Parallelism choice and manifest. Each pod runs a light regex check over 50 rows, so I request and limit exactly 1 CPU per pod. With 2 nodes $\times$ 2 allocatable CPUs = 4 CPUs, at most $4/1 = 4$ pods can run at once, so `parallelism: 4` matches the real capacity. The Job therefore completes in two waves of 4 pods, and I set `backoffLimit: 0` because a crash in a validator is a real bug that should fail loudly rather than be retried silently.

Full Indexed Job manifest (`job.yaml`); the Downward API supplies `POD_NAME` (`metadata.name`) and `NODE_NAME` (`spec.nodeName`). The `topologySpreadConstraints` block (`maxSkew: 1` over `kubernetes.io/hostname`) places 2 pods on each node per wave, matching the assumed 2 CPUs per node.

Evidence that parallelism 4 was reached. Right after `kubectl apply -f job.yaml`, four pods were Running at the same time in each wave (wave 1: shards 0–3, wave 2: shards 4–7), split 2 + 2 across the two nodes, and the Job ended with `COMPLETIONS 8/8` after 80s. Can be seen in the Video Recording.

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
shard-validator-0-sgqv2	1/1	Running	0	10s	10.244.1.2	minikube-m02
shard-validator-1-zl8pr	1/1	Running	0	10s	10.244.0.3	minikube
shard-validator-2-8zq4l	1/1	Running	0	10s	10.244.1.3	minikube-m02
shard-validator-3-zlqrp	1/1	Running	0	10s	10.244.0.4	minikube

NAME	STATUS	COMPLETIONS	DURATION	AGE
shard-validator	Complete	8/8	80s	2m44s

Excerpt of `kubectl get pods -o wide -w` (wave 1, shards 0–3, **Running** events sorted by shard). Wave 2 (shards 4–7) also had 4 pods **Running** at once, again split 2 + 2: pods 4 and 5 on minikube, pods 6 and 7 on minikube-m02.

Collecting results via the Kubernetes API. A shared volume would not work here because minikube’s default storage provisioner creates `hostPath` volumes on one node’s local disk, so pods on `minikube` and `minikube-m02` would write to different, unshared disks and no network filesystem is configured. Instead each pod prints its result to stdout, and `collect_results.py` uses the Kubernetes Python client (`list_namespaced_pod` with label `job-name=shard-validator`, then `read_namespaced_pod_log`) to gather all 8 results centrally; they match the seeded ground truth for every shard .

Shard	0	1	2	3	4	5	6	7
Seeded invalid rows (ground truth)	3	6	3	5	2	6	5	3
Invalid rows collected via API	3	6	3	5	2	6	5	3
Ran on node (m = minikube)	m02	m	m02	m	m	m	m02	m02

Would 3 nodes × 2 CPUs (6 total) change my choice? Yes, I would raise `parallelism` to 6, because 6 CPUs hold six 1-CPU pods at once, whereas `parallelism: 8` would need 8 CPUs and leave 2 pods **Pending**. For 8 equal-length shards this is still two waves (6+2), so I would not expect a shorter wall-clock time than 4+4 with my uniform 20s pods. The benefit is that the extra CPUs are actually used, and the Job finishes sooner whenever real shards take unequal time.

Question 4 — Kubernetes Deployments: Self-Healing & Rolling Updates

Self-healing evidence:

```
$ kubectl delete pod spam-api-6f8c6f6598-7qqll
pod "spam-api-6f8c6f6598-7qqll" deleted
$ kubectl get pods -l app=spam-api -w
spam-api-6f8c6f6598-8cpkv    0/1    Pending           0    0s
spam-api-6f8c6f6598-8cpkv    0/1    ContainerCreating 0    0s
spam-api-6f8c6f6598-8cpkv    1/1    Running           0    25s
```

A new pod (8cpkv) was created automatically at age 0s and reached **Running** within 25 seconds. This recovery is performed by the **ReplicaSet** controller that the Deployment manages, it continuously compares the desired replica count (2) against the actual healthy pod count and creates a replacement the moment that count drops below target.

Rolling update evidence:

```
$ kubectl set image deployment/spam-api spam-api=spam-api:1.1
deployment.apps/spam-api image updated

$ kubectl rollout status deployment/spam-api
Waiting: 1 out of 2 new replicas have been updated...
Waiting: 1 old replicas are pending termination...
deployment "spam-api" successfully rolled out

$ kubectl rollout history deployment/spam-api
REVISION  CHANGE-CAUSE
1          <none>
2          <none>
```

A background `curl` loop against `/healthz` kept returning 200 OK throughout, transitioning from 1.0 to 1.1 with no gaps, while `rollout status/history` confirm success across two revisions. As a Job instead, there would be no rolling-update mechanism: changing the image would mean deleting and recreating the Job, causing a full outage rather than this zero-downtime replacement.

Deployment vs. Job contrast: A Deployment reconciles continuously toward a target replica count, giving self-healing and rolling updates suited to long-running serving workloads. A Job runs pods to completion once and never restarts them, fitting run-to-completion batch work like Question 3. Swapping either for the other’s workload would be wrong: a Deployment would never let batch pods finish and exit, a Job offers no recovery or update path for an ongoing service.